TeleCon firmware sketches

Which `.ino` to flash

ESP32 DevKit over Bluetooth, and ESP32-CAM for video

Version 1.0 2026

Abstract

The Android app downloads one ZIP per connection from the firmware repository. Each ZIP is one Arduino sketch, and the sketch is the `.ino` with the same name. This note names every Bluetooth control sketch and every ESP32-CAM sketch, and the two ways to add a camera.

DevKit + Bluetooth CAM video only + DevKit One CAM: video and control

Contents

1 Two ways to add a camera **1**

2 Bluetooth control on an ESP32 DevKit **2**

3 ESP32-CAM, video only **3**

4 ESP32-CAM, control and camera on one board **3**

5 ESP32 DevKit over Wi-Fi, no camera **4**

6 What the ZIP contains **4**

1 Two ways to add a camera

Sticks, buttons, and telemetry always need a control sketch. Video is optional and uses an ESP32-CAM. Pick one of these layouts.

Layout	Boards	Sketches
Bluetooth only	One ESP32 DevKit	One of the three Bluetooth `.ino` files
Camera only, plus Bluetooth	ESP32-CAM for video, DevKit for sticks	`TeleCon_CAM_SoftAP_Video.ino` on the CAM, and one Bluetooth sketch on the DevKit
Control and camera together	One ESP32-CAM	`TeleCon_CAM_WiFi_Simple.ino` or `TeleCon_CAM_WiFi_Binary.ino`

The phone joins the camera SoftAP for the picture. It does not send sticks over that video link. Sticks go to the control sketch: Bluetooth on the DevKit, or TCP on the single CAM that also serves video.

2 Bluetooth control on an ESP32 DevKit

Board in the app: **DevKit**. No camera on this board. Open the sketch folder in the Arduino IDE (the folder that contains the `.ino`) and upload to an ESP32 Dev Module.

ZIP / `.ino`	Phone setting	On the air
`TeleCon_Classic_Simp`	Classic + Simple	Name `ESP32-TC-RC-BT-Simple`, PIN `1234`. Handshake `proto,simple`. Text lines `RC:CTRL`, `RC:BTN`, `RC:DATA`, `RC:PLOT`.
`TeleCon_Classic_Bina`	Classic + Binary	Name `ESP32-TC-RC-BT-Binary`. Handshake `proto,binary`. Frames `AA 55`, `BB 66`, `CC 11/22/33`.
`TeleCon_BLE_Binary`	BLE Binary	Name `ESP32-TC-RC-BLE-Binary` (Nordic UART). Same binary frames as Classic Binary.

`TeleCon_Classic_Simple.ino` Single file for a first board. Put motor and servo code in `onControl()` and `onButton()` where the file says `YOUR CODE HERE`. A steering servo on GPIO 13 is already wired in the sketch: stick `rx` $= 0$ is a 1500 µs pulse. Eight telemetry bytes (phone CH1–CH8) are chosen in `kTelemetryChannels[]` in this same file. With `ENABLE_PLOT` at 1 the board sends `RC:PLOT` about every 50 ms.

`TeleCon_Classic_Binary.ino` The `.ino` only starts Bluetooth and the vehicle modules. Pins, PWM, and the eight telemetry channels live in the other files in that folder. Edit

`pins.h` for your driver and `TelemetryChannels.h` for what CH1–CH8 report. The phone setting must be Classic + Binary before connect. The wire format is the control protocol document, not the text lines of the Simple sketch.

`TeleCon_BLE_Binary.ino` Same vehicle modules and the same `TelemetryChannels.h` table as Classic Binary. The link is BLE instead of Classic SPP. Choose BLE Binary in the app. Do not pair it as a Classic device.

These three sketches are also the control half of the two-board camera layout below. Flash only one of them on the DevKit.

3 ESP32-CAM, video only

Use this when a DevKit already runs one of the Bluetooth sketches and you want a picture from a second board.

`TeleCon_CAM_SoftAP_Video.ino` ZIP `TeleCon_CAM_SoftAP_Video.zip`. Flash an ESP32-CAM (AI-Thinker). This sketch serves video and does not accept sticks.

- SoftAP `TeleCon-RC-CAM-Starter`, password `telecon1234`, address `192.168.4.1`.
- Picture: `http://192.168.4.1/stream` (MJPEG) or `/capture` (one JPEG).
- Stream quality: `http://192.168.4.1/camconfig`.
- No control port. There is no `RC:CONNECT` on this board.

On the phone, turn on the camera overlay and keep the Bluetooth connection pointed at the DevKit (Classic Simple, Classic Binary, or BLE Binary). The phone uses Wi-Fi for the CAM and Bluetooth for the sticks at the same time. Do not flash `TeleCon_CAM_WiFi_Simple` for this layout: that sketch is the single-board case.

4 ESP32-CAM, control and camera on one board

One ESP32-CAM does both jobs. There is no second DevKit and no Bluetooth on this board. Control is TCP on the same SoftAP as the video. These sketches are for **RC Vehicle Pro** with board **CAM**.

ZIP / `.ino`	Phone setting	On the air
`TeleCon_CAM_WiFi_Simp`	Board CAM, Wi-Fi CAM Starter	SoftAP `TeleCon-RC-CAM-Starter` / `telecon1234`. Video on `192.168.4.1`. Sticks on TCP `192.168.4.1:3333`, handshake `proto,simple`.
`TeleCon_CAM_WiFi_Bina`	Board CAM, Wi-Fi SoftAP Binary	SoftAP `TeleCon-RC-CAM` / `telecon1234`. Same video host. Sticks on TCP port 3333, handshake `proto,binary`.

TeleCon_CAM_WiFi_Simple.ino Text control, same idea as Classic Simple, plus the camera stream. Camera pins stay free: put motors in `onControl()` and `onButton()` on pins that the camera is not using. The file points at the binary CAM sketch when you outgrow the single file.

TeleCon_CAM_WiFi_Binary.ino Binary frames (`AA 55`, `BB 66`, `CC 11/22/33`) and video on one CAM. `CameraStream` owns the SoftAP. The `.ino` starts Wi-Fi control, the vehicle modules, and the camera. With `TELECON_VEHICLE_IO` left at 0, camera pins stay free and motor outputs stay off until you enable IO and move those pins.

5 ESP32 DevKit over Wi-Fi, no camera

TeleCon_WiFi_Binary.ino ZIP `TeleCon_WiFi_Binary.zip`. This is not a Bluetooth sketch and it has no camera. The board is its own SoftAP `ESP32-TC-RC-WiFi-Binary`, password `telecon1234`. Sticks use binary frames on TCP `192.168.4.1:3333`, handshake `proto,binary`. In the app choose board DevKit and Wi-Fi SoftAP Binary. The plot still sends four samples on this sketch; the eight-channel table is only on the Bluetooth sketches.

6 What the ZIP contains

The app downloads `zips/TeleCon_<name>.zip`. Unzip it and open the folder that contains the `.ino` of the same name. Arduino compiles every `.cpp` and `.h` in that folder, so a binary sketch is the `.ino` plus those modules. Do not copy `SteerCenter` back in: stock sketches do not store a steering center. Trim stays on the phone, and `rx` $= 0$ is 1500 µs.